The review conclusion was
“Because the values are exactly cubic, they cannot be measured in silicon.”

That conclusion is not logically valid. The mistake is that they treated **“perfect polynomial shape”** as a sufficient proof of **“formula-generated data”** It is not. It is only a warning sign that needs investigation.

Now let’s explain why.

Imagine a small embedded benchmark where the program executes exactly the same instruction sequence every time for a given **d**. With

- No cache misses
- No OS scheduler
- Interrupts disabled
- No data-dependent branches
- No variable memory allocation
- No variable loops.

In that case, a real DWT cycle counter can absolutely **return the same number every run**. Embedded microcontrollers are not like Linux processes on a laptop. On a bare-metal Cortex-M4, **deterministic code can be extremely deterministic.** The code written for this study is exactly that kind of code.

The masked gadgets use randomness, yes. But randomness is used as data, not as control flow. That distinction is the heart of the issue.

In **isw_and**, the number of loop iterations is determined by **n**, not by **the random word**. The code calls rand32(), XORs and ANDs with shares, and stores results. But whether the random word is 0x12345678 or 0xDEADBEEF, the same instructions execute. The same is true for **refresh_sni**, **inject_uniform**, and **linear_refresh**. The loops are based on **n**, and random values are consumed inside fixed loops.

So we can inferred

- Random values do not automatically imply random timing.
  They only create timing variation if they affect something timing-relevant like
  - Branches,
  - Loop bounds,
  - Memory addresses,
  - Wait states,
  - Variable-latency instructions,
  - or Peripheral stalls.

In this code, the random values affect the masked output, but they do not choose different execution paths.

Now look at the sequential algorithm itself.
The **a2b_batch32()** processes shares from s = 0 to s < n. For each share, it transposes 32 coefficients over **KS_WORDS**, then calls **inject_uniform**. For every share after the first, it calls **ks_add()**. Inside **ks_add()**, the main cost comes from **repeated isw_and() calls**. **isw_and()** is **quadratic** in the number of shares because it has the nested loop:

```
for i = 0..n-1
   for j = i+1..n-1
```

That is about $\frac{n(n-1)}{2}$ random-consuming pair operations.
So we can summarize all facts as below

- **a2b_batch32()** sequentially accumulates across shares. It calls expensive additions repeatedly as s grows.
- **ks_add()** has many **isw_and()** calls.
- **isw_and()** is quadratic in **n**.

Therefore the total sequential **cost** naturally becomes a **low-degree polynomial in n or d.**

A **cubic-looking** sequence is therefore not surprising. It is exactly the kind of shape you expect from nested fixed loops. So this cubic-looking does not prove: **"The data was generated by a fake formula".**
Why? Because a real program with nested fixed loops can also have a polynomial instruction count.
Think of this simple example:

```
for (i = 0; i < d; i++)
 for (j = 0; j < d; j++)
   for (k = 0; k < d; k++)
     do_fixed_work();
```

If you measure this on a **bare-metal MCU** with **interrupts disabled**, the cycle count can be exactly:

$$a.d^3 + b.d^2 + c.d + e$$

That does not mean the result is fake. It means the code structure is cubic.

Now let’s address the review’s strongest assumption: the TRNG.

A stress test design to figure out the performance of M4 TRNG module Actively.

```
void rng_direct_stress_test_v2(void) {
 cyccnt_init();
 rng_init_direct();

 uart_write("BEGIN_RNG_STRESS_V2\r\n");
 uart_write("idx,sr_before,value,sr_after,spins_until_next_ready,cycles_until_"
            "next_ready,error_flags_after\r\n");

 for (uint32_t i = 0; i < 64; i++) {
   /*
    * Wait for the current word to be ready.
    */
   while ((RNG->SR & RNG_SR_DRDY) == 0u) {
   }

   uint32_t sr_before = RNG->SR;
   uint32_t value = RNG->DR;
   uint32_t sr_after = RNG->SR;

   /*
    * Now test whether DRDY clears and how long it takes to become ready again.
    */
   uint32_t spins = 0u;

   uint32_t t0 = cyccnt_read();

   while ((RNG->SR & RNG_SR_DRDY) == 0u) {
     spins++;
   }

   uint32_t t1 = cyccnt_read();

   uint32_t error_flags_after = RNG->SR & (RNG_SR_CECS | RNG_SR_SECS);

   g_rng_sink ^= value;

   uart_write_u32(i);
   uart_write_char(',');
```

```
    uart_write_hex32(sr_before);
    uart_write_char(',');

    uart_write_hex32(value);
    uart_write_char(',');

    uart_write_hex32(sr_after);
    uart_write_char(',');

    uart_write_u32(spins);
    uart_write_char(',');

    uart_write_u32(t1 - t0);
    uart_write_char(',');

    uart_write_hex32(error_flags_after);
    uart_write("\r\n");
  }

  uart_write("END_RNG_STRESS_V2\r\n");
}
```

It is assumed that because the benchmark calls the STM32F407 hardware RNG many times, the timing must contain physical variability. That sounds logical at first, but this test shows it is not what actually happened. The raw result of this test can be found in the table below.

id	sr_before	value	sr_after	spins_until_next_ready	cycles_until_next_ready	error_flags_after
0	0x00000001	0x529C16B6	0x00000000	1	16	0x00000000
1	0x00000001	0xDB412516	0x00000000	1	16	0x00000000
2	0x00000001	0x3725EF4D	0x00000000	1	16	0x00000000
3	0x00000001	0x2B874FF8	0x00000000	1	16	0x00000000
4	0x00000001	0x78E8D412	0x00000000	1	16	0x00000000
5	0x00000001	0xEF53E585	0x00000000	1	16	0x00000000
6	0x00000001	0x5E2DC81B	0x00000000	1	16	0x00000000
7	0x00000001	0x9EAD555E	0x00000000	1	16	0x00000000
8	0x00000001	0xE9D898F1	0x00000000	1	16	0x00000000
9	0x00000001	0xD8BA778F	0x00000000	1	16	0x00000000
10	0x00000001	0x1BCE7CBC	0x00000000	1	16	0x00000000
11	0x00000001	0x9F54AE73	0x00000000	1	16	0x00000000

12	0x00000001	0x54F6E0A0	0x00000000	1	16	0x00000000
13	0x00000001	0x6B70D17E	0x00000000	1	16	0x00000000
14	0x00000001	0x49B0925E	0x00000000	1	16	0x00000000
15	0x00000001	0xDAABF1BB	0x00000000	1	16	0x00000000
16	0x00000001	0xDC0EAEAB	0x00000000	1	16	0x00000000
17	0x00000001	0x346798DE	0x00000000	1	16	0x00000000
18	0x00000001	0x8698AC3B	0x00000000	1	16	0x00000000
19	0x00000001	0xA5B6A7A3	0x00000000	1	16	0x00000000
20	0x00000001	0x379FC403	0x00000000	1	16	0x00000000
21	0x00000001	0xE22FDB9B	0x00000000	1	16	0x00000000
22	0x00000001	0x223ABABB	0x00000000	1	16	0x00000000
23	0x00000001	0xB57EA79B	0x00000000	1	16	0x00000000
24	0x00000001	0xAE40B36D	0x00000000	1	16	0x00000000
25	0x00000001	0xBB4A0ECC	0x00000000	1	16	0x00000000
26	0x00000001	0xCC9B49A0	0x00000000	1	16	0x00000000
27	0x00000001	0x7A7A4D2A	0x00000000	1	16	0x00000000
28	0x00000001	0x83878412	0x00000000	1	16	0x00000000
29	0x00000001	0x7DB7BD05	0x00000000	1	16	0x00000000
30	0x00000001	0xCEF67BA7	0x00000000	1	16	0x00000000
31	0x00000001	0x6417934F	0x00000000	1	16	0x00000000
32	0x00000001	0xBCFBA861	0x00000000	1	16	0x00000000
33	0x00000001	0xE04661E8	0x00000000	1	16	0x00000000
34	0x00000001	0xF0CEB704	0x00000000	1	16	0x00000000
35	0x00000001	0x1995CAD2	0x00000000	1	16	0x00000000
36	0x00000001	0xEE4D9ECE	0x00000000	1	16	0x00000000
37	0x00000001	0x788C3F36	0x00000000	1	16	0x00000000
38	0x00000001	0x3993AD84	0x00000000	1	16	0x00000000
39	0x00000001	0x43760CF2	0x00000000	1	16	0x00000000
40	0x00000001	0xDD1ECA19	0x00000000	1	16	0x00000000
41	0x00000001	0xD9DA0742	0x00000000	1	16	0x00000000
42	0x00000001	0xAB184A4B	0x00000000	1	16	0x00000000
43	0x00000001	0xF926B0D4	0x00000000	1	16	0x00000000
44	0x00000001	0xC3AAB2A5	0x00000000	1	16	0x00000000

45	0x00000001	0x539B3EE2	0x00000000	1	16	0x00000000
46	0x00000001	0xD4465A99	0x00000000	1	16	0x00000000
47	0x00000001	0xCAA7F6F3	0x00000000	1	16	0x00000000
48	0x00000001	0x019D4BDE	0x00000000	1	16	0x00000000
49	0x00000001	0x19D9EA51	0x00000000	1	16	0x00000000
50	0x00000001	0xEC4C60A1	0x00000000	1	16	0x00000000
51	0x00000001	0x5EFFCC9C	0x00000000	1	16	0x00000000
52	0x00000001	0x1EB95F0B	0x00000000	1	16	0x00000000
53	0x00000001	0xAA87D561	0x00000000	1	16	0x00000000
54	0x00000001	0x0DE732E8	0x00000000	1	16	0x00000000
55	0x00000001	0xC5DBFEF5	0x00000000	1	16	0x00000000
56	0x00000001	0x27B7474F	0x00000000	1	16	0x00000000
57	0x00000001	0x87615404	0x00000000	1	16	0x00000000
58	0x00000001	0xC447F9DC	0x00000000	1	16	0x00000000
59	0x00000001	0x4EFBDB06	0x00000000	1	16	0x00000000
60	0x00000001	0x02E051BB	0x00000000	1	16	0x00000000
61	0x00000001	0x8D8C2A2A	0x00000000	1	16	0x00000000
62	0x00000001	0x1B1239B6	0x00000000	1	16	0x00000000
63	0x00000001	0x91A7E60F	0x00000000	1	16	0x00000000

By considering this result the below inference given

```
sr_before = 0x00000001
sr_after  = 0x00000000
spins_until_next_ready = 1
cycles_until_next_ready = 16
error_flags_after = 0x00000000
```

Clearly below logic concluded

- sr_before = 1 means DRDY is set before the read: a random word is ready.
- sr_after = 0 means reading RNG->DR clears the ready flag: the register behavior is real, not fake or stuck.
- value changes every time: the RNG is not returning a constant.
- error_flags_after = 0: no RNG clock or seed error is reported.
- spins_until_next_ready = 1: the next word becomes ready after one polling iteration in this setup.

So the TRNG is active, but its observed refill/read behavior is highly stable in this benchmark.